IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA

(DEEMED TO BE UNIVERSITY)

Project –Low Level Design

on

Personalized Recommendation Engine for Retail

Student Name: Niha Najeeb

Industry Mentor: Sumit Shukla

Guided By: Ankitha

Table of Contents

1. Introduction

1.1 Scope of the document

This document provides a detailed Low-Level Design (LLD) of the Retail Recommendation Dashboard system. It focuses on the internal implementation details, including module-level architecture, data structures, algorithms, and execution flow.

- The scope includes:
- Internal working of recommendation logic
- Module interactions and responsibilities
- Data transformation and processing
- UI rendering mechanism

1.2 Intended Audience

This document is intended for:

- Developers implementing or modifying the system
- Students learning recommender systems
- Evaluators assessing technical depth
- System architects for further enhancements

1.3 System Overview

The system is a **content-based recommendation engine** designed for retail products. It suggests similar products based on textual attributes (tags) such as category, usage, and features.

The system uses:

- **CountVectorizer** for feature extraction
- **Cosine Similarity** for similarity computation

The output is displayed as an interactive shopping dashboard using product cards.

2. System Design

2.1 Application Design

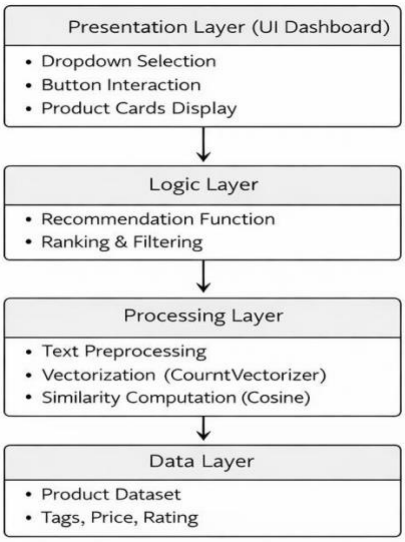

Figure 2.1: Application Design Diagram

At the implementation level, the application consists of tightly integrated modules:

- **Data Initialization Module:**
  Loads dataset into a Pandas DataFrame and ensures correct schema.
- **Preprocessing Module:**
  Handles text normalization such as lowercase conversion, tokenization, and removal of inconsistencies.
- **Feature Extraction Module:**
  Converts processed text into numerical vectors using CountVectorizer.
- **Similarity Computation Module:**
  Generates similarity matrix using cosine similarity.
- **Recommendation Engine:**
  Applies ranking logic and retrieves top-N similar products.
- **UI Rendering Module:**
  Uses ipywidgets and HTML to display results in card format

2.2 Process Flow

Figure 2.2: Process Flow Diagram

Detailed execution pipeline:

1. System initializes and loads dataset
2. Tags are normalized (lowercase, cleaned)
3. Vocabulary is created using CountVectorizer
4. Tags are transformed into sparse matrix
5. Cosine similarity matrix is computed (NxN matrix)
6. User selects a product from dropdown
7. Product index is retrieved using DataFrame lookup
8. Corresponding similarity scores are fetched
9. Scores are sorted in descending order
10. Top-N products (excluding itself) are selected
11. Results are formatted for display
12. UI renders product cards with price, rating, and similarity

2.3 Information Flow

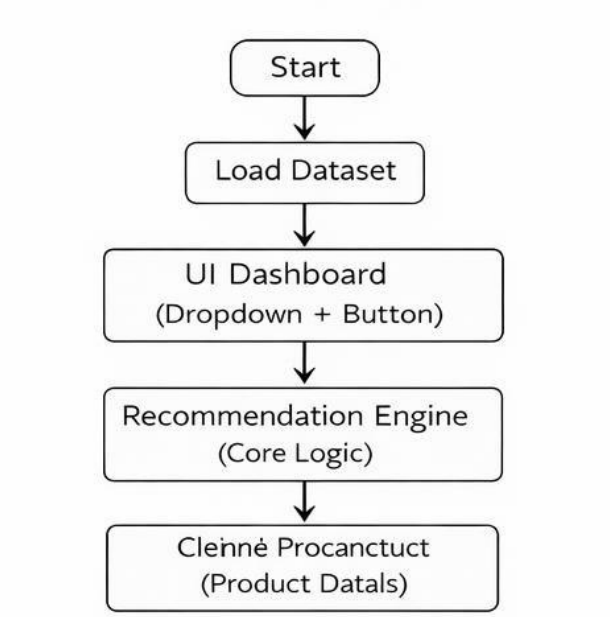

Figure 2.3: Information Flow Diagram

Internal data flow:

- User Input → Dropdown selection
- Input passed to recommendation function
- Function queries dataset for index
- Index used to retrieve similarity scores
- Scores processed and ranked
- Result data structured into display format
- Output passed to UI renderer
- UI displays product cards dynamically

2.4 Components Design

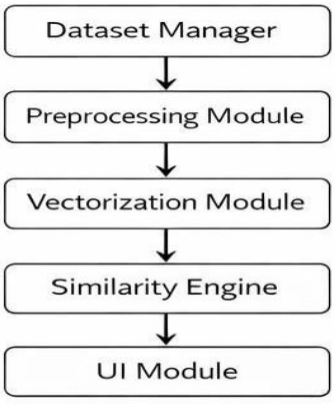

Figure 2.4: Components Design Diagram

1. Dataset Manager

- Stores product data in DataFrame
- Provides access methods (filter, index, retrieval)
- Ensures data consistency

2. Preprocessing Module

- Converts text to lowercase
- Removes unwanted characters
- Standardizes tag format
- Handles missing values (if any)

3. Vectorization Module

- Uses CountVectorizer
- Converts text into Bag-of-Words representation
- Generates feature matrix

4. Similarity Engine

- Computes cosine similarity between vectors
- Produces similarity matrix (NxN)
- Stores matrix in memory for quick access

5. Recommendation Module

- Retrieves similarity scores for selected item
- Sorts items based on similarity
- Excludes self-comparison
- Applies fallback logic if similarity is zero
- Returns top-N recommendations

6. UI Module

- Implements dropdown and button using ipywidgets
- Displays output using HTML templates
- Generates product cards dynamically
- Handles user interaction events

2.5 Key Design Considerations

- **Efficiency:** Use vectorized operations for fast computation
- **Scalability:** Modular design supports dataset expansion
- **Reusability:** Components can be reused across domains
- **User Experience:** Interactive UI improves usability
- **Maintainability:** Clean separation of modules

2.6 API Catalogue

API Name	Description	Input	Output
load_data()	Loads dataset into memory	File path	DataFrame
preprocess_data()	Cleans and prepares tags	Raw data	Processed data
vectorize_data()	Converts tags to vectors	Tags	Feature matrix
compute_similarity()	Generates similarity matrix	Vectors	Matrix
get_index()	Fetch product index	Product name	Index

recommend_product()	Returns recommendations	Product name	List of items
render_cards()	Displays product cards	Data	HTML output

3. Data Design

3.1 Data Model

Field Name Type Description item String Product name tags Text Descriptive features price Integer Product price rating Float Product rating

3.2 Data Access Mechanism

- Data is accessed via Pandas DataFrame
- Uses indexing (iloc, loc) for retrieval
- Filtering based on conditions
- In-memory operations for speed

3.3 Data Retention Policies

- Data stored locally in memory
- No persistent storage required
- Dataset can be modified or updated dynamically
- No user data stored

3.4 Data Migration

- Supports CSV import/export
- Can integrate with SQL databases
- Can scale to cloud storage in future

4. Interfaces

- **User Interface:**
  Interactive dashboard (dropdown, buttons, product cards)
- **System Interface:**
  Python modules communicate via function calls
- **External Interface:**
  Future API integration for real-time product data

5. State and Session Management

- System is stateless
- Each request processed independently
- No session tracking implemented
- No user history stored

6. Caching

- No caching currently
- Possible improvement:
- Cache similarity matrix
- Store frequently accessed results

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data handling
- Input validation for user inputs
- Safe execution environment
- No external exposure

7.2 Performance Aspects

- Efficient vectorized computation
- Low latency for recommendation generation
- Suitable for small to medium datasets
- Memory usage optimized

8. References

- Python Official Documentation
- Pandas Documentation
- Scikit-learn Documentation
- Research papers on recommender systems
- Online tutorials and academic resources